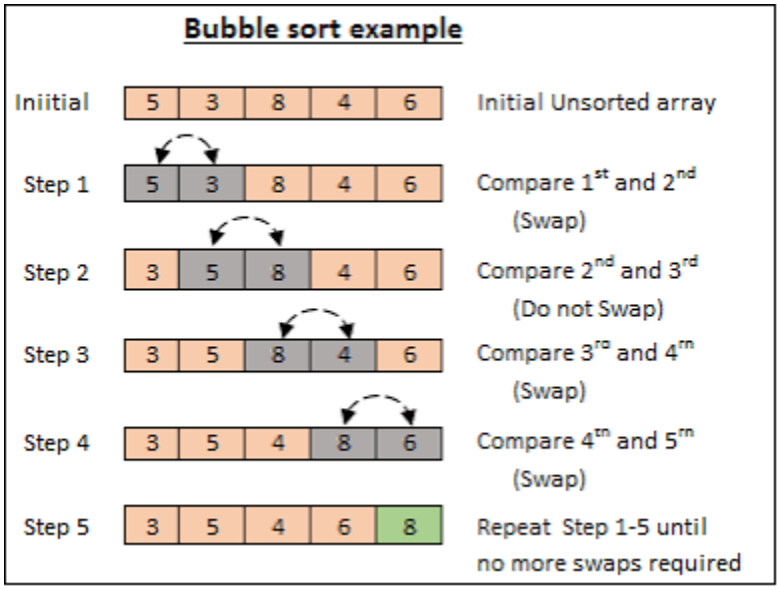
1. Bubble Sort



Definition: Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Working:

- Compare adjacent elements.
- Swap if the left element is greater than the right element.
- After each pass, the largest unsorted element reaches the end.
- Repeat until the array is sorted.

Java code

```
public static void bubbleSort(int arr[]) {
    int n = arr.length;

    for (int turn = 0; turn < n - 1; turn++) {
        for (int j = 0; j < n - 1 - turn; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Example: `[5, 3, 4, 1]`

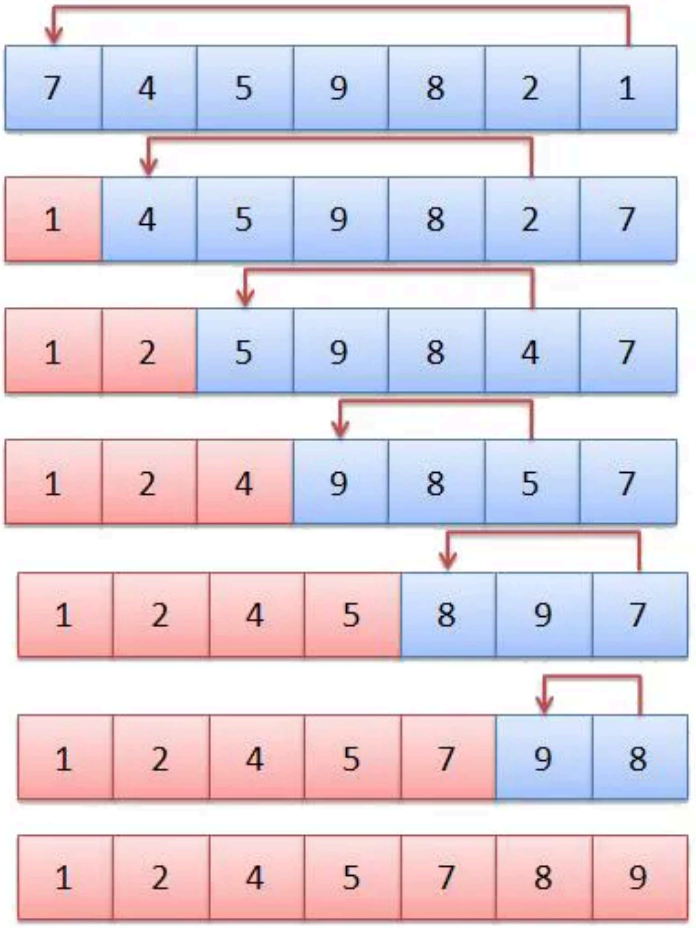
- Pass 1: `[3, 4, 1, 5]`
- Pass 2: `[3, 1, 4, 5]`
- Pass 3: `[1, 3, 4, 5]`

Time complexity

- Best: $O(n)$ with an early-stop optimization.
- Average: $O(n^2)$
- Worst: $O(n^2)$
- Space: $O(1)$

Key point: Largest element moves to the end in each pass.

2. Selection Sort



Definition: Selection Sort finds the minimum element from the unsorted portion and swaps it with the first element of that portion.

Working:

- Assume the first unsorted element is the minimum.
- Search the remaining unsorted elements for a smaller value.
- Store its index in `minPos`.
- Swap the minimum with the first unsorted element.
- Repeat for the remaining array.

Java code

```
public static void selectionSort(int arr[]) {
    int n = arr.length;

    for (int i = 0; i < n - 1; i++) {
        int minPos = i;

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minPos]) {
                minPos = j;
            }
        }

        int temp = arr[i];
        arr[i] = arr[minPos];
        arr[minPos] = temp;
    }
}
```

Example: `[5, 3, 4, 1]`

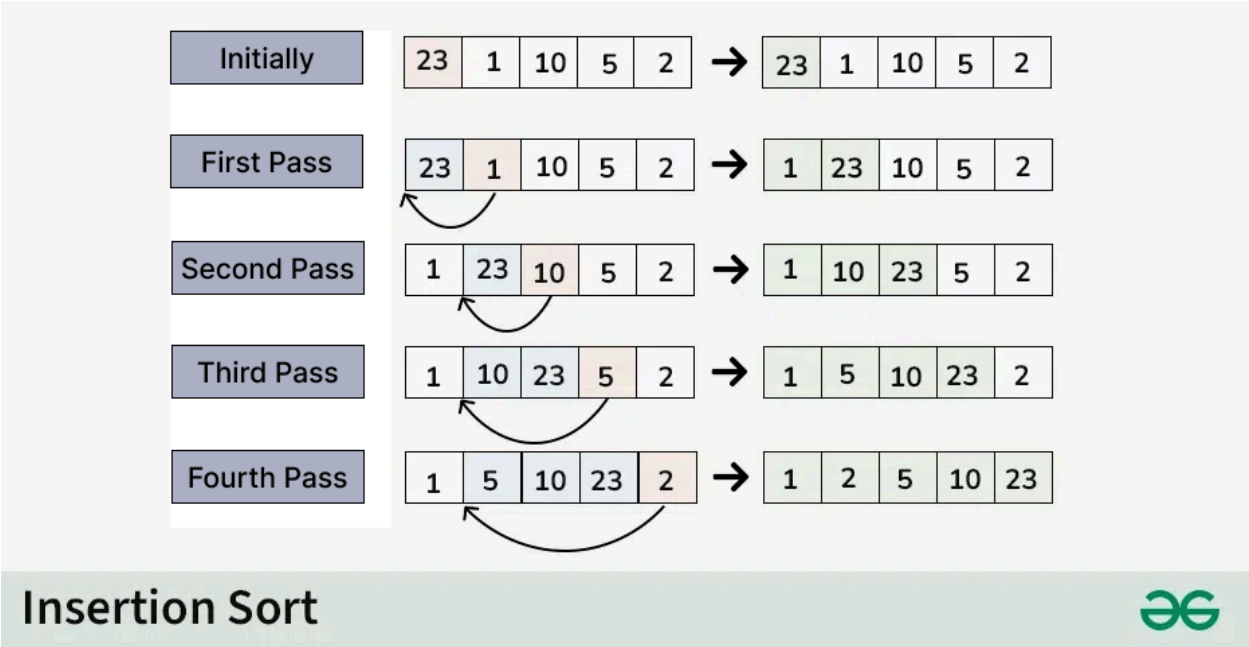
- Pass 1: `[1, 3, 4, 5]`
- Pass 2: `[1, 3, 4, 5]`
- Pass 3: `[1, 3, 4, 5]`

Time complexity

- Best: $O(n^2)$
- Average: $O(n^2)$
- Worst: $O(n^2)$
- Space: $O(1)$

Key point: Minimum element moves to the beginning in each pass. Selection Sort performs at most $n - 1$ swaps.

3. Insertion Sort



1. Definition

Insertion Sort is a sorting algorithm that builds a sorted array one element at a time by inserting each element into its correct position.

It works like arranging playing cards in your hand.

2. Algorithm

1. Start from index `1`, assuming the first element is already sorted.
2. Store the current element in `curr`.
3. Set `prev = i - 1` to point to the previous element.
4. Compare `arr[prev]` with `curr`.
5. If `arr[prev] > curr`, shift the previous element one position to the right.
6. Decrement `prev` and repeat while `prev >= 0` and the previous element is greater than `curr`.
7. Insert `curr` at `arr[prev + 1]`.
8. Repeat until the array is sorted.

3. Java code

```
class Main {
    public static void insertionSort(int arr[]) {
        for (int i = 1; i < arr.length; i++) {
            int curr = arr[i]; // Current element to insert
            int prev = i - 1;  // Index of previous element

            while (prev >= 0 && arr[prev] > curr) {
                arr[prev + 1] = arr[prev]; // Shift larger element right
                prev--;
            }

            arr[prev + 1] = curr; // Insert at correct position
        }
    }

    public static void printArr(int arr[]) {
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
```

4. Important points for exams and interviews

- Insertion Sort is an in-place sorting algorithm.
- It is stable because equal elements are not shifted past one another when using `arr[prev] > curr`.
- It is adaptive because it performs efficiently on nearly sorted arrays.
- The left portion remains sorted after each pass.
- For an array of length n, the outer loop executes at most $n - 1$ times.
- For descending order, change `arr[prev] > curr` to `arr[prev] < curr`.

Memory trick: `curr` stores the element, `prev` searches left, shifting makes space, and `prev + 1` is the insertion position.

Java Built-in Methods for Arrays

1. `Arrays.sort()` — Ascending order

Sorts an array from smallest to largest.

```
import java.util.Arrays;


int arr[] = {5, 2, 4, 1, 3};


Arrays.sort(arr);


System.out.println(Arrays.toString(arr));
```

Output:

```
[1, 2, 3, 4, 5]
```

Time complexity: Typically $O(n \log n)$ for primitive arrays.

2. Sort an array in descending order

For descending order, use the wrapper class `Integer[]` instead of primitive `int[]`.

```
import java.util.Arrays;

import java.util.Collections;


Integer arr[] = {5, 2, 4, 1, 3};


Arrays.sort(arr, Collections.reverseOrder());


System.out.println(Arrays.toString(arr));
```

Output:

```
[5, 4, 3, 2, 1]
```

Remember: `Collections.reverseOrder()` cannot be used directly with primitive `int[]`.

3. Reverse an array

Java does not provide a direct `Arrays.reverse()` method. You can reverse an array using two pointers.

```
int arr[] = {1, 2, 3, 4, 5};


int left = 0;

int right = arr.length - 1;


while (left < right) {

    int temp = arr[left];

    arr[left] = arr[right];

    arr[right] = temp;


    left++;

    right--;

}
```

Output:

[5, 4, 3, 2, 1]

Time complexity: $O(n)$

Counting Sort — Java Notes

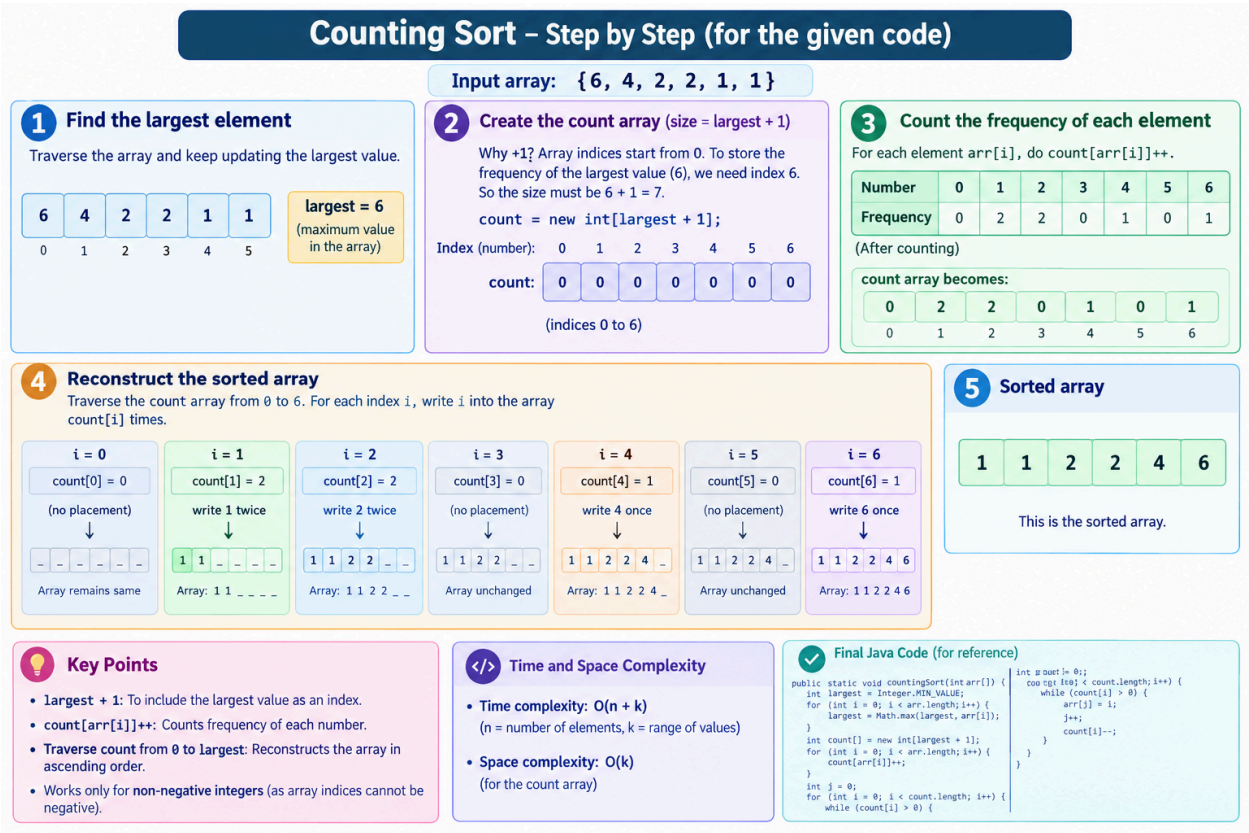
1. Definition

Counting Sort is a non-comparison-based sorting algorithm that sorts elements by counting the frequency of each value.

2. Algorithm

1. Find the largest element in the array.
2. Create a `count` array of size `largest + 1`.
3. Count the frequency of every element using `count[arr[i]]++`.
4. Traverse the `count` array from index `0` to `largest`.
5. Place each number into the original array according to its frequency.

3. Counting sort




```
public static void countingSort(int arr[]) {
    int largest = Integer.MIN_VALUE;


    // Find the largest element
    for (int i = 0; i < arr.length; i++) {
        largest = Math.max(largest, arr[i]);
```

```
    }


    // +1 because array indices start from 0
    // and we need index 'largest' also
    int count[] = new int[largest + 1];


    // Store the frequency of each element
    for (int i = 0; i < arr.length; i++) {
        Count[arr[i]]++; //if 1 occurs 2 times in unsorted array then the frequency of the number 1 will be updated to 2 at index 1
    }


    int j = 0;


    // Reconstruct the array in ascending order
    for (int i = 0; i < count.length; i++) {
        while (count[i] > 0) {//if count is 0 that means that number is not present in the unsorted array
            arr[j] = i;
            j++;
            count[i]--;//frequency decreases
        }
    }
}
```

4. Dry run

Input: `[6, 4, 2, 2, 1, 1]`

Frequency array:

Number (index)	Frequency
0	0
1	2
2	2
3	0
4	1
5	0
6	1

Traverse the frequency array in ascending order and write each number according to its frequency.

Output: `[1, 1, 2, 2, 4, 6]`

5. Why `largest + 1`?

If `largest = 6`, we need indices `0, 1, 2, 3, 4, 5, 6`.

That is 7 positions, so:

int count[] = new int[largest + 1];

Without `+1`, index `6` would be out of bounds.

6. Time and space complexity

Let n be the number of elements and k be the range of values from `0` to `largest`.

Case	Complexity
Best time	$O(n + k)$
Average time	$O(n + k)$
Worst time	$O(n + k)$
Auxiliary space	$O(k)$

7. Important points

- Counting Sort does not compare elements with one another.
- It works efficiently when the value range is small.
- It supports duplicate elements.
- This implementation assumes non-negative integers.
- If the largest value is very large compared with the array size, memory usage can be excessive.
- The basic frequency-array implementation shown here is not stable in its reconstruction form, though Counting Sort can be made stable using cumulative counts and an output array.

Memory trick: Find the largest → Count frequencies → Reconstruct the sorted array.